Experimenting with Unsupervised Machine Learning Algorithms

Guneet Kaur Kapur
Supervised by: Dr. Anasse Bari

5.2 Abstract

This paper works with unsupervised machine learning algorithms as a tool for predictive analytics through direct experimentation and changing parameter choices. This paper reviews four main algorithms: K-means, K-means++, Hierarchical Clustering, and K-nearest neighbors. It covers how each works mathematically, advantages, disadvantages, and example applications. K-means is then applied to the Mall Customer Segmentation dataset, a 200 person set of customer demographics. With this algorithm, every resulting cluster must be interpreted qualitatively rather than scored quantitatively. For experimentation, four parameters were changed against a baseline of k=5 run on both annual income and spending score: adding another parameter (age), changing the number of clusters (k=8), Euclidean vs. Manhattan Distance, and the initialization strategy (utilizing K-means++). Adding age as a third feature produced the largest change because it was able to split the baseline's largest cluster into a younger and an older segment that could not have been separated with income and spending alone. Increasing k broke existing clusters into near-duplicates rather than surfacing new customer types which established k=5 as the best cluster amount. Substituting Manhattan distance for Euclidean distance did not change the output at all. K-means++ initialization reproduced the random-initialization result within a single customer.

5.3 Introduction

Predictive analytics is the practice of using machine learning algorithms to uncover patterns and use those to make predictions. It allows you to take existing data and extract information and data from it that can create huge marketing and economic opportunities. Using past data, a situation is quickly analyzed and used to forecast future outcomes.

Unsupervised Learning (data clustering) is a type of machine learning where an algorithm discovers hidden patterns, structures, or relationships in data independently. Unsupervised learning is a tool that allows us to practice using Predictive Analytics. It is a type of machine learning where an algorithm discovers hidden patterns, structures, or relationships in data without any human guidance or pre-existing labels. This is both the method's strength and its main difficulty: because nobody defined the groups, the algorithm can surface patterns no analyst would have thought to look for, but there is also no answer key to confirm that what it found is real.

By contrast, supervised learning (data classification) is when the machine learning model trains on labeled data, meaning every example has a corresponding correct answer. The process has three stages. First, data collection and labeling, in which examples are gathered and each is tagged with its known result. Second, training where the model is shown those examples repeatedly and adjusts internally to narrow the gap between its predictions and the true labels which also to model building. Third, testing, where the trained model is evaluated on data it has never seen. This is a check on whether it learned a general rule or memorized its training set.

Although supervised and unsupervised learning are different types of machine learning tools, they can be used in tandem to make a product better. A prime example of this intersection is in OpenAI, or familiarly known as, ChatGPT. ChatGPT is trained using a combination of supervised and unsupervised learning to come together in a technique called Reinforcement Learning from Human Feedback (RLHF). First, OpenAI uses a base model that uses vast amounts of different texts from the Internet. It learns structure, grammar, and context of language by predicting the next word in a sequence on its own [48].

However, an example where they do not work in tandem is demonstrated in healthcare. In healthcare, a supervised approach trains on chest X-rays labeled "tumor" or "no tumor" to learn diagnosis; an unsupervised approach clusters patient data to discover disease subtypes that no prior medical hypothesis proposed. That second healthcare example is not hypothetical. Seymour et al. applied k-means clustering to 29 clinical variables across 20,189 sepsis patients at twelve Pennsylvania hospitals, identifying four clinical phenotypes that correlated with distinct host-response patterns and clinical outcomes, with simulations suggesting these groupings help explain why the same treatment produces different results in different patients [49]. Sepsis had long been treated as one syndrome; clustering suggested it behaves as several.

In Marketing, predictive analytics can be used for dynamic churn prediction. Companies use survival analysis, utilizing predictive analytics, to flag customers showing microscopic behavioral changes before they leave the company. A real-world example of this is Hydrant, a wellness brand. Hydrant applies predictive modeling to tracking customer purchases [we need a reference]. They track

patterns in customer behavior before they cancel their subscription, allowing them to maximize customer engagement.

5.4 Machine Learning Algorithms

There are four different types of machine learning algorithms we will be focusing on.

K-means

K-means is a centroid-based, iterative partitioning machine learning algorithm that takes unlabelled data and sorts it into 'k' number of distinct groups [1]. The data is in its most natural form and it does not have "correct answers" associated with it for the model to use in its training [2]. The algorithm works in four main steps. First, the algorithm creates k number of groups/clusters. Each group is represented by a randomly chosen data point and the point serves as a "centroid" [1]. These centroids represent the initial data cluster centers [1,3]. Second, the remaining data points are grouped into clusters based on the centroid they are most similar to [1,5]. This is calculated using Squared Euclidean Distance to measure similarity and the Argmin Operator to select the closest centroid. [1, 7]. For the i-th feature, x_i is the value of the data point, c_i is the value of the centroid, and n is the total number of features [7].

$$d(X, C) = \sqrt{\sum_{i=1}^{n} (x_i - c_i)^2}$$

Third, all of the data points in each cluster are added up and averaged using the Arithmetic Mean Vector (shown below). These numbers are what the algorithm uses to recalculate new centroids ($\mu_k^{(t+1)}$) for new clusters [1, 5, 15]. During iteration t, C is the set of all data points currently assigned to cluster k and the summation is the sum of every other number.

$$\mu_k^{(t+1)} = \frac{1}{|C^{(t)}|} \sum_{x_i \in C^{(t)}} x_i$$

That brings us to the last and most important step: iteration [3]. The algorithm repeats the process until the centroids stay the same for multiple iterations in a row or the limit of possible iterations has been done [1].

K-Means Clustering is popular because it is easy to use for beginners [1, 3]. The concept of partitioning data based on distance is easy to understand using multiple coding languages. Second, it is easily scalable to accommodate large datasets while still keeping its performance consistent [1, 3, 6]. Third, the clusters that it finds are fairly easy to interpret, which is essential when making business decisions based on the algorithm [1, 6].

However, there are still disadvantages to this machine learning algorithm. One of the most important problems with K-means is the "random initialization trap" where poorly chosen random starting points can lead to suboptimal groupings or unstable results [11]. The clusters formed need to be near spherical in shape and all of the clusters need to be roughly equal in size [8, 10]. Oftentimes, with raw data this can be hard to find. It also does not account well for outliers in the data [8, 10]. The algorithm defines clusters using mean values, so the centroid values can be easily skewed, leading to poor cluster arrangements and inaccurate results [8, 10].

It is best to use K-means clustering when you have data that is low-dimensional and has continuous numerical data that can easily separate into evenly sized clusters [1, 12]. If these requirements are not met, the data would need to be prepared beforehand.

Three of the most common applications for this algorithm are customer segmentation, image compression, and anomaly detection [1]. Within customer segmentation, this algorithm is often used by retailers to send personalized advertisements to customers [1, 13]. It groups customers into clusters based on shared traits, scaling features so the algorithm is not affected by outliers [1, 13]. For image compression, this algorithm is used to reduce an image file size by drastically decreasing the number of unique colors in the image [1, 14]. This allows the image to be stored and transmitted much more efficiently while preserving its visual quality [1, 14]. Finally, in anomaly detection, the algorithm is used to identify data points that act as outliers by flagging items that fall too far away from any major cluster center [1].

K-means++

K-means++ is an algorithm which is used to select the centroids discussed above for K-Means clustering [16]. It was created in 2007 to solve the "random initialization trap" in K-means [17]. To do this, K-means++ spreads out the initial centroids before the main clustering process begins [9, 16]. It greatly reduces the number of iterations needed to finish the K-means process [9]. The algorithm works in three main steps. It starts by choosing the first cluster center uniformly at random from the data points ($\frac{1}{n}\ for\ all\ i$) to ensure an unbiased starting position [20]. Second, it calculates the distance from each data point to the center picked first using the Euclidean distance formula (with a slight modification on c) [16, 20, 21].

$$D(x) = \sqrt{\sum_{i=1}^{n} (x_i - c_1)^2}$$

The algorithm then squares D(x), making it clearer which points are farther away, making it more likely they are chosen as the next centroid [16]. Third, it divides each point's squared distance by the total sum of distances to assign a probability to each point [16].

$$P(x) = \frac{D(x)^2}{\sum D(x)^2}$$

The points closer get a lower P(x) than those farther away and a random weighted selection chooses the next centers based on those probabilities [16]. For subsequent centers, D(x) updates to represent the distance to the nearest chosen centroid [18, 19]. Once that is complete, the standard K-means algorithm can continue.

The main advantage of this algorithm is that it gets K-means out of the "random initialization trap" with bad starting points [9, 23, 24]. This leads to faster learning and better results when K-means is run [24]. Studies show that K-means++ improves overall clustering accuracy dramatically, avoiding grouping errors like separating a big cluster [24].

The disadvantages for K-means++ are essentially the same as those for the K-Means algorithm as it is meant to be more of a helpful tool. Key drawbacks of this algorithm include that it has a slower setup time, assumes round clusters, and is easily confused by outliers [9, 24].

K-means++ is generally used in the same applications as K-means but another common application includes fraud detection [23]. Within fraud detection, it is fairly common for banks to use this algorithm to spot theft. For example, if a credit card is suddenly used in a new country for a large purchase, the algorithm spots it as falling out of its normal "cluster" and triggers a warning [23].

Hierarchical Clustering

Hierarchical clustering is an unsupervised machine learning algorithm that groups similar data points into a tree-like hierarchy of clusters [33]. It does not require a number that specifies the number of clusters (like the above algorithms) making it a good choice for exploratory data analysis [33, 34]. It builds a dendrogram that helps visualize relationships between the data and uses that to decide the best number of clusters [33, 34]. A dendrogram is like a "family tree." The lower the merge point, the more similar the groups are [33, 35].

There are two main approaches to hierarchical clustering: Agglomerative with a "bottom to up" approach and Divisive with a "top to down" approach [36].

Agglomerative clustering starts with individual data points and each data point acts as its own "cluster" [36]. Then, the distance is calculated between every pair of clusters using either the Euclidean Distance, Manhattan Distance, or Cosine Distance [37]. Euclidean Distance is the straight-line distance between two points, Manhattan Distance is the sum of the absolute differences between the coordinates, and Cosine Distance is the angular difference between vectors [37]. These are used initially when you are simply measuring point to point.

$$\text{Euclidean: } D(x) = \sqrt{\sum_{i=1}^{n} (x_i - c_1)^2}$$

$$\text{Manhattan: } D(x_i, j_i) = \sum_{k=1}^{d} |x_{ik} - x_{jk}|$$

$$\text{Cosine: } D(x_i, j_i) = 1 - \frac{x_i \times j_i}{||x_i||\,||x_j||}$$

As the algorithm continues on, group-to-group distances start to be measured, requiring linkage equations [38, 39]. There are four different types of linkages: Single, Complete, Average, and Ward's [40]. Different linkage equation choices completely change how clusters form [40].

Third, the algorithm finds and combines the two clusters that have the smallest linkage distance. Fourth, instead of calculating distances between every single data point again, the algorithm updates the distance matrix using the Lance–Williams recurrence formula. This updates the distance between the newly combined cluster and any remaining cluster using the distances already calculated in earlier steps.

Another type of Hierarchical Clustering is Divisive Clustering. Divisive clustering starts with all the data points in one single cluster. Then, the cluster is divided into two smaller clusters by taking the most isolated point and forming a new "splinter group," pulling in other points closest to it.

For each new cluster, this splitting process repeats recursively until every data point sits in its own individual cluster. To calculate the distance between the two data points, the same metrics as used for Agglomerative Clustering are used.

This type of unsupervised machine learning algorithm offers many key advantages for data analysis. One of its biggest advantages is its flexibility- you do not need to guess the number of groups in advance and it provides deep insights into the data's structure [41]. Another major advantage is that, unlike the earlier algorithms, it is able to work with complex, not uniform shapes [42]. It is able to find oddly shaped clusters, not just circular [42]. Also, due to the fact that it produces a tree diagram, it is easy to visualize the results the dendrogram is showing [38, 43].

However, there are key disadvantages. Within the process, if any mistakes are made in earlier steps, they become permanently locked and cannot be undone [43, 44]. Once two points are merged, they can never be separated later even if better groupings are found [44]. One wrong result can potentially mess up the entirety of the algorithm [44]. Second, the algorithm runs too slowly for large datasets. The algorithm calculates the distance between every single data point and with thousands of data points, it takes a long time [45]. Third, outliers skew the results (just as with every other algorithm looked at above) [45].

This machine learning algorithm is commonly used in customer segmentation and gene expression analysis [46, 47]. Retailers (like Target) group shoppers by their preferences and target users with discounts based on their favorite drink for example [46]. Second, medical researchers often use this to analyze genetic data to find groups of genes that act similarly under conditions such as cancer [46, 47].

K Nearest Neighbors

K-Nearest Neighbors (KNN) is a machine learning algorithm (unsupervised) that classifies new data or predicts continuous values based on known data points [26]. When a new data point is introduced, the algorithm looks at the "K" number of nearest data points near it. Based on what the majority are categorized as, the algorithm sorts it into that respective cluster/category [26]. It is considered a "lazy" learning algorithm because it doesn't have a strict equation to go on during the training process, rather it memorizes the training dataset and performs its calculations when requested [27]. The first step for this algorithm is the same as that of K-means, choose the k-value, the number of "neighbors" to analyze [28]. Second, the distance between the new data point and every other point in the dataset is measured using either Euclidean or Manhattan distance (shown above with Hierarchical Clustering) [26, 28]. Third, use a sorting algorithm to organize the smallest to largest distances from the data point and take the top K data points for the algorithm [28]. If the algorithm is being used for classification, then based on what the majority are categorized as, the algorithm sorts it into that respective cluster/category [26]. If the algorithm is being used for regression, the average value of the neighbors are taken to predict the new data points number [26, 28].

One key advantage of this algorithm is that it makes no assumptions about the data it's given, making it a non-parametric algorithm [29, 30]. It is highly versatile, naturally multi-class and has instant adaptability [29, 30]. This means that it can effectively handle different variations of tasks, scales to multi-class problems without transformations, and new training data can be added dynamically without having to retrain the entire model [29, 30]. Some key disadvantages include the high computational cost during prediction, heavy memory requirements, and inaccurate performance in high-dimensional datasets [29, 30].

KNN is used across industries for multiple tasks such as data similarity, proximity, and historical pattern matching. First, it is mainly used in recommendation engines such as for platforms like Netflix, Amazon, and Spotify to auto-recommend items in real time [31]. Second, it is often used in the financial industry for credit score and fraud detection. It evaluates credit risk by comparing a new applicant’s financial history to past customers [32]

5.5 Experimental Results

The "Mall Customer Segmentation" dataset [] is made up of the demographics of a small mall's customers with 200 rows/records. Each row represents one customer. There are 5 different attributes: CustomerID (int), Gender (categorical), Age (int), Annual Income (int), Spending Score (int). There is no ground-truth label because this dataset does not contain a labelled "correct answer" as K-means, an unsupervised machine learning algorithm, is meant to be.

Varying parameters changes the effect of the clusters formed. We will be working with adding another parameter (Age), changing the number of clusters formed (k-value), changing the initialization strategy (using k-means++), and changing the distance metrics. By isolating these metrics and comparing it to the baseline shown above, the effects of changing each of these parameters can be utilized.

For each of the following iterations, in order for accurate comparison, we kept the random state seed (for the initial locations of centroids) the same throughout (except when using K-means++ to initialize).

The first iteration of the algorithm was only run with 2 of the 5 columns: Annual Income and Spending Score. CustomerID and Gender were excluded because they were not numerical features that added to the goal of the clustering. Age was also excluded in this iteration of the dataset.

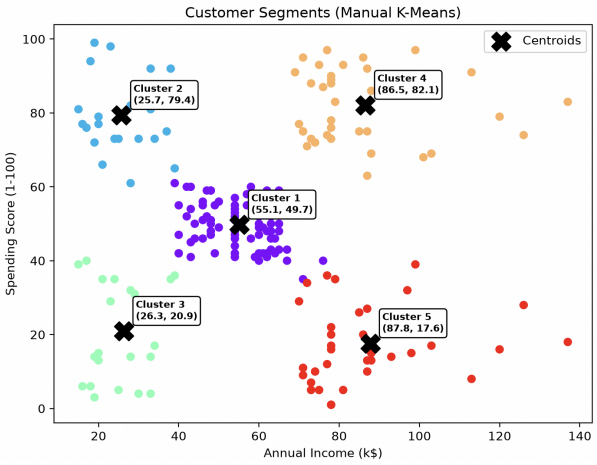


Figure 1: Scatterplot with labelled clusters and centroids formed by the K-means algorithm

With a value of k=5, Table 1 and Figure 1 exemplify the results of the first iteration of the K-means algorithm. Since the Mall Customer Segmentation dataset does not contain ground-truth labels, all five clusters are going to be qualitatively interpreted. Table 1 summarizes the five clusters produced with k=5.

Cluster 1 had the highest number of customers (n=80) and represented customers who had moderate income, around $55,000, and a moderate spending score of ~50. This cluster had nearly three times as many people as clusters 2 and 3, and nearly twice as many people as clusters 4 and 5.

Cluster	Annual Income (k$)	Spending Score (1-100)	Number of data points
1	55.087500	49.712500	80
2	25.727273	79.363636	22
3	26.304348	20.913043	23
4	86.538462	82.128205	39
5	87.750000	17.583333	36

Table 1: Clusters formed by K-means algorithm

Cluster 2 was composed of 22 customers and represented low income customers with a moderately high spending score. This shows that these customers tend to spend a lot in this particular mall. This shows a bit of an unreliable or aspirational spending in proportion to their income.

Cluster 3 captured 23 customers with relatively low income and a low spending score. This demonstrates more conscious spending with more of a budget.

Cluster 4 represents approximately 39 high-income mall customers with relative high income and a high spending score. This is the ideal customer that the mall is looking for because they tend to spend money there often, and likely large amounts of it.

Cluster 5 represents 36 high-income customers with a relatively low spending score. These customers have strong purchasing power but they do not tend to spend at the mall.

Clusters 2 and 3 have income on the lower side but have highly different spending scores and clusters 4 and 5 are mirrors of that, except with higher income.

When the K-means algorithm with the same conditions as the previous iteration with an added parameter of age, the following results occur:



Figure 2: 3D Scatterplot with labelled clusters and centroids formed by the K-means algorithm with an added parameter

Cluster 1 represents 40 relatively young (age = ~32) who have both high-income and tend to spend a lot at this particular mall.

Cluster 2 represents 54 new adults (age = ~25) who have relatively low-income and moderately high spending. This represents a new group not necessarily seen in the baseline. It shows young customers spending relatively freely in comparison to their income.

Cluster 3 represents 20 older adults (age = ~46) who have both low income and low-spending. This matches the baseline Cluster 3, making it more specific to older adults.

Cluster 4 represents 39 older adults (age = ~39) who have high income and low spending. This matches the baseline Cluster 5, making it more specific to middle-aged people.

Cluster	Age	Annual Income (k$)	Spending Score (1-100)	Count
1	32.875000	86.100000	81.525000	40
2	25.185185	41.092593	62.240741	54
3	46.250000	26.750000	18.350000	20
4	39.871795	86.102564	19.358974	39
5	55.638298	54.382979	48.851064	47

Table 2: Clusters formed by K-means algorithm

Cluster 5 represents 47 adults, the highest age group of around 55 years old with both moderate income and moderate spending. Compared to the baseline, it looks like it absorbed part of the older Cluster 1 with specifically the older mainstream customers.

Adding age as another dimension allowed us to look at the data in a closer way because it split some of the baseline clusters by age. Notably, the baseline Cluster 1 was split into this new (Cluster 2, n=54) and an older sub-group (Cluster 5, n=47), suggesting Age does meaningfully differentiate customers who otherwise looked identical on income/spending alone.

To test the accuracy of the clusters with a higher k-value, I tested the algorithm with k=7 and k=15. With a value of k=8, the K-means algorithm was run and concluded with the data shown in Table 3 and Figure 3.

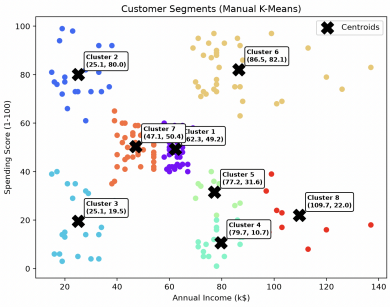


Figure 3: 2D Scatterplot with labelled clusters and centroids formed by the K-means algorithm with k = 7

Cluster	Annual Income (k$)	Spending Score (1-100)	Count
1	62.297297	49.243243	37
2	25.095238	80.047619	21
3	25.142857	19.523810	21
4	79.684211	10.736842	19
5	77.222222	31.555556	9
6	86.538462	82.128205	39
7	47.068182	50.363636	44
8	109.700000	22.000000	10

Table 3: Clusters formed by K-means algorithm

Cluster 1 represents 37 customers with moderate income (~$62k) and a moderate spending score (~49). Similar to baseline Cluster 1 but smaller and skewed slightly higher-income.

Cluster 2 represents 21 customers with low income (~$25k) and a high spending score (~80). Matches baseline Cluster 2, aspirational spenders who spend more than they should.

Cluster 3 represents 21 customers with low income (~$25k) and a low spending score (~20). Matches baseline Cluster 3, more budget-conscious and less engagement customers.

Cluster 4 represents 19 customers with high income (~$80k) and a very low spending score (~11). Even though they have purchasing power, this group barely spends at all at this mall despite strong purchasing power.

Cluster 5 represents 9 customers with high income (~$77k) and low-moderate spending (~32). This is a small cluster that appears to have split off from the baseline's high-income/low-spending group.

Cluster 6 represents 39 customers with high income (~$87k) and a high spending score (~82). Matches baseline Cluster 4 almost exactly the "ideal customer" segment.

Cluster 7 represents 44 customers with moderate income (~$47k) and a moderate spending score (~50). This is identical in profile to Cluster 1, just a second "mainstream" cluster.

Cluster 8 represents 10 customers with the highest income in the dataset (~$110k) and a low spending score (~22). A newly isolated segment of very high earners who spend little — too small a group to appear as its own cluster at k=5, but distinct enough to split out once k increases.

Clusters 1 and 7 are nearly indistinguishable (both moderate/moderate), and Clusters 4, 5, and 8 all represent shades of "high income, low spending," just split by degree. This confirms the earlier point — increasing k mostly fragmented existing groups rather than revealing new customer types.

For all previous iterations, Euclidean distance was used to calculate the distance between the points. In this iteration, we will replace Euclidean distance with Manhattan distance and observe the difference in performance.

Cluster 1 represents 80 customers, moderate income (~$55k) and moderate spending (~50) — identical to baseline Cluster 1.

Cluster 2 represents 22 customers, low income (~$26k) and high spending (~79) — identical to baseline Cluster 2.

Cluster 3 represents 23 customers, low income (~$26k) and low spending (~21) — identical to baseline Cluster 3.

Cluster 4 represents 36 customers, high income (~$88k) and low spending (~18) — identical to baseline Cluster 5.

Cluster 5 represents 39 customers, high income (~$87k) and high spending (~82) — identical to baseline Cluster 4.

Every value matches the Euclidean baseline exactly, down to the decimal. Switching distance metrics had zero effect on this dataset's clustering — a sign the five groups are extremely well-separated regardless of how distance is measured. This is more specific to this dataset rather than another.

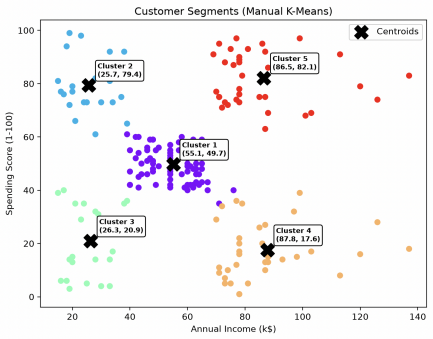


Figure 4: 2D Scatterplot with labelled clusters and centroids formed by the K-means algorithm with Manhattan Distance metrics

Cluster	Annual Income (k$)	Spending Score (1-100)	Count
1	55.087500	49.712500	80
2	25.727273	79.363636	22
3	26.304348	20.913043	23
4	87.750000	17.583333	36
5	86.538462	82.128205	39

Table 4: Clusters formed by K-means algorithm

Changing the initialization process from pure random initialization to utilizing K-means is different

Cluster 1 represents 81 customers, moderate income (~$55k) and moderate spending (~50) — matches baseline Cluster 1, with one additional customer.

Cluster 2 represents 35 customers, high income (~$88k) and low spending (~17) — matches baseline Cluster 5, with one fewer customer.

Cluster 3 represents 22 customers, low income (~$26k) and high spending (~79) — identical to baseline Cluster 2.

Cluster 4 represents 39 customers, high income (~$87k) and high spending (~82) — identical to baseline Cluster 4.

Cluster 5 represents 23 customers, low income (~$26k) and low spending (~21) — identical to baseline Cluster 3.

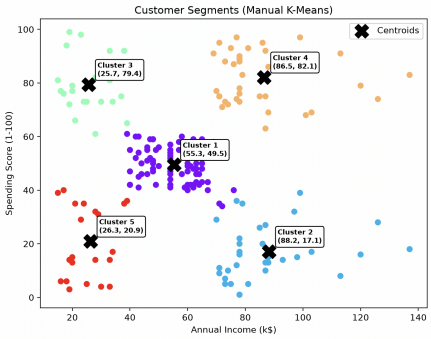


Figure 5: 2D Scatterplot with labelled clusters and centroids formed using K-means++ and the K-means algorithm

Cluster	Annual Income (k$)	Spending Score (1-100)	Count
1	55.296296	49.518519	81
2	88.200000	17.114286	35
3	25.727273	79.363636	22
4	86.538462	82.128205	39
5	26.304348	20.913043	23

Table 5: Clusters formed by K-means++ and K-means

5.6 Discussions and Conclusions

Across all four experiments, not all the changes in parameters had significant consequences. Changing the number of clusters, changing the distance metric, and changing the initialization strategy left the underlying structure of the data essentially intact.

Changing k. Increasing the number of clusters did not reveal new customer segments; it just divided the ones already present. At the higher k value, two clusters were nearly indistinguishable from one another, both describing moderate-income, moderate-spending customers. The high-income/low-spending group also broke into multiple, smaller groups separated by degree rather than by any behavioral difference a marketer could act on. This points to k=5 being close to the natural pattern of the data. It also highlights something new about K-means: because the algorithm always produces exactly k clusters, it will always produce them, whether or not k genuinely distinct groups exist.

Changing the distance metric. Replacing Euclidean distance with Manhattan distance produced no change at all. This shows that the five segments are separated by margins wide enough that the math used to measure distance for this dataset becomes irrelevant. Points tend to sit near one centroid than any other under both distance metrics. This finding is a property of this dataset rather than a general result, so more testing would need to be done on more complex datasets

Changing the initialization strategy. K-means++ produced clusters nearly identical to plain random initialization, differing by the assignment of a single customer. K-means++ is designed to reduce the variance of outcomes across many runs by spreading initial centroids apart and avoiding the random initialization trap. A realization here is that it does not promise a different answer on any particular run, especially on data this well-separated, where most starting configurations converge to the same solution regardless.

Adding another feature. This was the most consequential change. Where the other three parameters resized or relabeled existing groups, adding age restructured them. The baseline's largest cluster was split into a younger sub-segment and an older sub-segment that income and spending alone had identified as one big cluster (being identical). Age carried real information that the original two features did not contain, which means the baseline clustering was under-specified.

Limitations. First, the most significant of the limitations is the lack of quantitative validation. Because the dataset carries no ground-truth labels, every cluster was interpreted qualitatively. The claim that k=5 therefore rests on judgment based on testing rather than measurement. Second, the initialization comparison used one seed (for consistency over multiple tests and parameter changes) which does not fully assess stability. Third, the findings rest on a single small dataset of 200 records with only three usable numerical features, which is both clean and unusually well-separated. These are abnormal conditions for datasets which are often messier and higher-dimensional. Results from data this cooperative cannot be generalized to noisier settings.

Future work. In the future, exploring more of the machine learning algorithms would allow a lot more conclusions to be drawn. For example, implementing hierarchical clustering on the same dataset would allow the dendrogram to suggest a cluster count independently and would let the four linkage

methods be compared directly against the K-means partitions. Second, repeating the full experiment on a larger and less cleanly separated dataset would establish how much of the stability observed here belongs to K-means and how much belongs to this particular set of 200 customers.

Unsupervised learning is often described as a method that finds structure without human guidance. These experiments suggest a narrower reading. The algorithm receives no labels, but it is still entirely dependent on human decisions about which features to give it, how many groups to ask for, and what the groups it returns actually mean.

5.7 References

[1] GeeksforGeeks. (2026, May 1). *K means Clustering – Introduction*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/
[2] GeeksforGeeks. (2025, July 23). *What is the difference between labeled and unlabeled data?* GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/what-is-the-difference-between-labeled-and-unlabeled-data/
[3] Ikotun, A. M., Ezugwu, A. E., Abualigah, L., Abuhaija, B., & Heming, J. (2022). K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. Information Sciences, 622, 178–210. https://doi.org/10.1016/j.ins.2022.11.139
[4]
[5] Dabbura, I. (2025, March 7). *K-Means Clustering: algorithm, applications, evaluation methods, and drawbacks*. Towards Data Science. https://towardsdatascience.com/k-means-clustering-algorithm-applications-evaluation-methods-and-drawbacks-aa03e644b48a/
[6] Bahmani, B., Moseley, B., Vattani, A., Kumar, R., & Vassilvitskii, S. (n.d.). *Scalable K-Means++*. Stanford. https://theory.stanford.edu/~sergei/papers/vldb12-kmpar.pdf
[7] GeeksforGeeks. (2026, July 29). Euclidean distance. GeeksforGeeks. https://www.geeksforgeeks.org/maths/euclidean-distance/
[8] Advantages and disadvantages of k-means. (n.d.). Google for Developers. https://developers.google.com/machine-learning/clustering/kmeans/advantages-disadvantages
[9] GeeksforGeeks. (2025, July 23). KMeans vs KMeans++ Clustering Algorithm. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/k-means-vs-k-means-clustering-algorithm/
[10] 365 Data Science. (2019, January 24). K means clustering: Pros and cons of K means clustering [Video]. YouTube. https://www.youtube.com/watch?v=YIGtalP1mv0
[11] GeeksforGeeks. (2021, June 8). ML | Random Initialization Trap in KMEANs. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/ml-random-intialization-trap-in-k-means/
[12] Nalinthasnai, N., Thammasudjarit, R., Tassaneyasin, T., Eksombatchai, D., Sungkanuparph, S., Boonsarngsuk, V., Sutherasan, Y., Junhasavasdikul, D., Theerawit, P., & Petnak, T. (2025). Unsupervised machine learning clustering approach for hospitalized COVID-19 pneumonia patients. BMC Pulmonary Medicine, 25(1), 70. https://doi.org/10.1186/s12890-025-03536-w
[13] Perapu, P. (2025). Customer segmentation using K-Means clustering for personalized marketing campaigns. International Journal of Scientific Research in Computer Science Engineering and Information Technology, 11(3), 810–815. https://doi.org/10.32628/cseit25113344

[14] GeeksforGeeks. (2025, July 11). Image compression using Kmeans clustering. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/image-compression-using-k-means-clustering/
[15] Leo, C. (2025, January 23). The math behind K-Means clustering. Towards Data Science. https://towardsdatascience.com/the-math-and-code-behind-k-means-clustering-795582423666/
[16] GeeksforGeeks. (2026, May 2). KMEANS++ Algorithm ML. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/ml-k-means-algorithm/
[17] Arthur, D., Vassilvitskii, S., Stanford University, & Stanford University. (n.d.). k-means++: The Advantages of Careful Seeding. In K-means++: The Advantages of Careful Seeding [Journal-article]. https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf
[18] Zhu, A. (2025, January 24). 7 Most asked questions on K-Means clustering. Towards Data Science. https://towardsdatascience.com/explain-ml-in-a-simple-way-k-means-clustering-e925d019743b/
[19] K-Means Clustering — H2O 3.46.0.11 documentation. (2023, April 27). https://docs.h2o.ai/h2o/latest-stable/h2o-docs/data-science/k-means.html
[20] kmeans - k-means clustering - MATLAB. (n.d.). https://www.mathworks.com/help/stats/kmeans.html
[21] Bhardwaj, H., & Srinivasa Rao, D. (2025). A MODIFIED K-MEANS APPROACH FOR EFFECTIVE CLUSTERING USING WEIGHTED ADJACENT MATRIX. In Medi-Caps University & Medi-Caps University, Journal of Theoretical and Applied Information Technology (Vol. 103, Issue 5). Little Lion Scientific. https://www.jatit.org/volumes/Vol103No5/24Vol103No5.pdf
[22] Mehta, A., Nair, P., Kaya, O., & Singh, V. (2025). Financial Fraud Detection through K-means Clustering-based unsupervised Learning. ResearchGate. https://www.researchgate.net/publication/394088510_Financial_Fraud_Detection_through_K-means_Clustering-based_unsupervised_Learning
[23] Massi, M. C., Ieva, F., & Lettieri, E. (2020). Data mining application to healthcare fraud detection: a two-step unsupervised clustering method for outlier detection with administrative databases. BMC Medical Informatics and Decision Making, 20(1), 160. https://doi.org/10.1186/s12911-020-01143-9
[24] Arthur, D., Vassilvitskii, S., Stanford University, & Stanford University. (n.d.-b). k-means++: The Advantages of Careful Seeding. In K-means++: The Advantages of Careful Seeding [Journal-article]. https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf
[25] Kavlakoglu, E. (2025, November 17). KNN. IBM. https://www.ibm.com/think/topics/knn
[26] GeeksforGeeks. (2026, May 2). KNeArest Neighbor(KNN) algorithm. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/
[27] GeeksforGeeks. (2024, November 12). Why is KNN a lazy learner? GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/why-is-knn-a-lazy-learner/
[28] Ai, T. A. (2024, October 18). K-Nearest Neighbor(KNN) algorithm in machine learning. Applied AI Blog. https://www.appliedaicourse.com/blog/knn-algorithm-in-machine-learning/

[29] Kavlakoglu, E. (2025, November 17). KNN. IBM. https://www.ibm.com/think/topics/knn
[30] GeeksforGeeks. (2025b, January 27). Pros and Cons Of using KNN [Video]. YouTube. https://www.youtube.com/watch?v=Ah1xPsEJBYs
[31] Baum, A. (2025, January 22). Example applications of K-Nearest-Neighbors. Towards Data Science. https://towardsdatascience.com/example-applications-of-k-nearest-neighbors-e6e47cd73f1f/
[32] Yin, L., Zhang, X., & Xiong, X. (2025). Application and effectiveness of KNN model in stock market prediction. Advances in Economics Management and Political Sciences, 182(1), 129–133. https://doi.org/10.54254/2754-1169/2024.23738
[33] GeeksforGeeks. (2026, April 16). Hierarchical Clustering in Machine Learning. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/hierarchical-clustering/
[34] Factory, F. A. (2026, July 20). What is hierarchical clustering? Benefits and examples. FPT AI. https://factory.fpt.ai/ai-insights/what-is-hierarchical-clustering
[35] 2.3. Clustering. (n.d.). Scikit-learn. https://scikit-learn.org/stable/modules/clustering.html
[36] GeeksforGeeks. (2025, May 15). Difference Between Agglomerative clustering and Divisive clustering. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/difference-between-agglomerative-clustering-and-divisive-clustering/
[37] GeeksforGeeks. (2025, July 23). Agglomerative clustering with different metrics in Scikit Learn. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/agglomerative-clustering-with-different-metrics-in-scikit-learn/
[38] Noble, J. (2025, November 17). Hierarchical clustering. IBM. https://www.ibm.com/think/topics/hierarchical-clustering
[39] **https://ijrpr.com/uploads/V4ISSUE4/IJRPR12087.pdf**
[40] GeeksforGeeks. (2025c, July 12). Types of linkages in hierarchical clustering. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/ml-types-of-linkages-in-clustering/
[41] MetricGate. (2025, May 28). Hierarchical Clustering (Agglomerative) calculator. MetricGate. https://metricgate.com/docs/hierarchical-clustering/
[42] GeeksforGeeks. (2025, July 12). Hierarchical clustering in data mining. GeeksforGeeks. https://www.geeksforgeeks.org/data-science/hierarchical-clustering-in-data-mining/
[43] Chen, J., MacEachren, A., & Peuquet, D. (2009). Constructing Overview + Detail Dendrogram-Matrix views. IEEE Transactions on Visualization and Computer Graphics, 15(6), 889–896. https://doi.org/10.1109/tvcg.2009.130
[44] Sultana, S. I. (2023, November 1). How the Hierarchical Clustering Algorithm Works. Dataaspirant. https://dataaspirant.com/hierarchical-clustering-algorithm/
[45] GeeksforGeeks. (2025, July 12). Hierarchical clustering in data mining. GeeksforGeeks. https://www.geeksforgeeks.org/data-science/hierarchical-clustering-in-data-mining/

[46] Bock, T. (2026, July 17). What is Hierarchical Clustering? Displayr. https://www.displayr.com/what-is-hierarchical-clustering/
[47] Microlearning Daily. (2024, June 9). Effective customer segmentation and clustering techniques (11 minutes) [Video]. YouTube. https://www.youtube.com/watch?v=AOe3hjo1wrc